

Reporte de Laboratorio S05

Paso 1 y 2: Clonar y Levantar con Compose

Cloné el repositorio en su versión v05.1 y construí los contenedores usando el comando `docker compose up --build -d`. Al verificar el estado con `docker compose ps`, me aseguré de que ambos servicios (db y app) estuvieran en estado `healthy`, garantizando que la base de datos y la aplicación arrancaron correctamente.

```
[+] up 5/5
✓ Image mexi-banco-app           Built
✓ Volume mexi-banco_mexibanco_datos Created
✓ Network mexibanco              Created
✓ Container mexi-banco-db-1      Healthy
✓ Container mexi-banco-app-1     Started
```

```
$ docker compose ps
NAME                IMAGE              COMMAND                                SERVICE  CREATED        STATUS
mexi-banco-app-1    mexi-banco-app    "java -jar app.jar"                  app      25 seconds ago Up 19 seconds (healthy)
mexi-banco-db-1     postgres:17-alpine "docker-entrypoint.s..."            db       25 seconds ago Up 24 seconds (healthy)
```

Paso 3: Prueba de Endpoints en Postman

Importé la colección descargada en la aplicación de escritorio de Postman y ejecuté las peticiones en orden. Pude comprobar el correcto funcionamiento del banco al abrir cuentas, realizar transferencias, simular un SPEI duplicado y consultar el estado de cuenta y notificaciones.

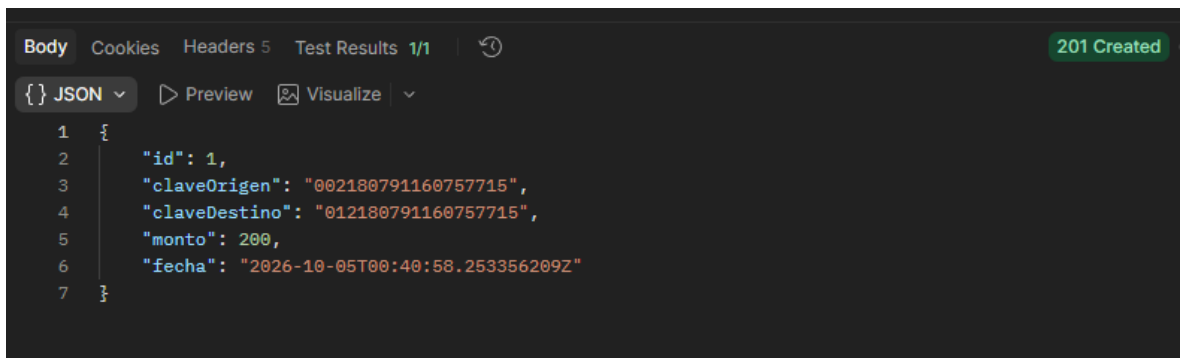
```
Body  Cookies  Headers 5  Test Results 2/2  201 Created
{ } JSON  Preview  Visualize
1  {
2    "id": 1,
3    "clabe": "002180791160757715",
4    "titular": "Ana",
5    "saldo": 1000
6  }
```

Abrir cuenta origen



```
Body Cookies Headers 5 Test Results 2/2
{} JSON Preview Visualize
1 {
2   "id": 2,
3   "clabe": "012180791158521436",
4   "titular": "Beto",
5   "saldo": 500
6 }
```

Abrir cuenta destino

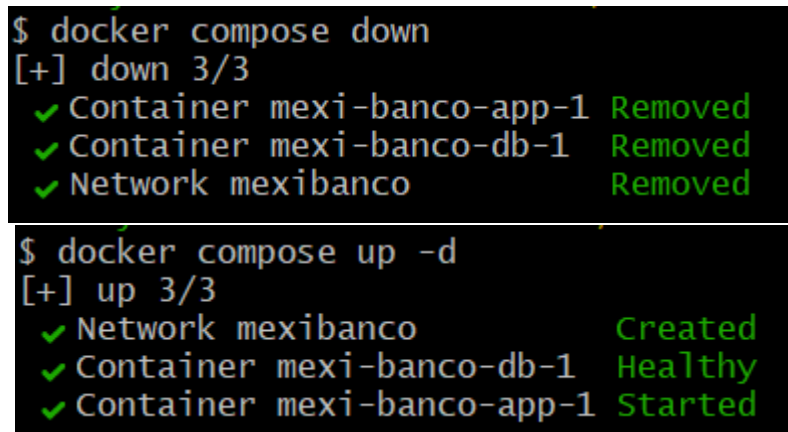


```
Body Cookies Headers 5 Test Results 1/1 201 Created
{} JSON Preview Visualize
1 {
2   "id": 1,
3   "claveOrigen": "002180791160757715",
4   "claveDestino": "012180791160757715",
5   "monto": 200,
6   "fecha": "2026-10-05T00:40:58.253356209Z"
7 }
```

Transferir 200 a cuenta destino

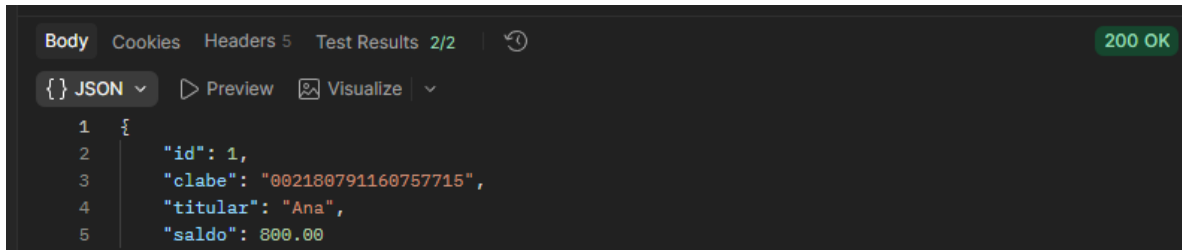
Paso 4: Bajar con y sin volumen

Al ejecutar docker compose down y volver a levantar el sistema, mis datos permanecieron intactos. Esto sucedió porque el comando apaga los contenedores y borra la red, pero conserva el volumen de la base de datos intacto.



```
$ docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed
✓ Container mexi-banco-db-1 Removed
✓ Network mexibanco Removed

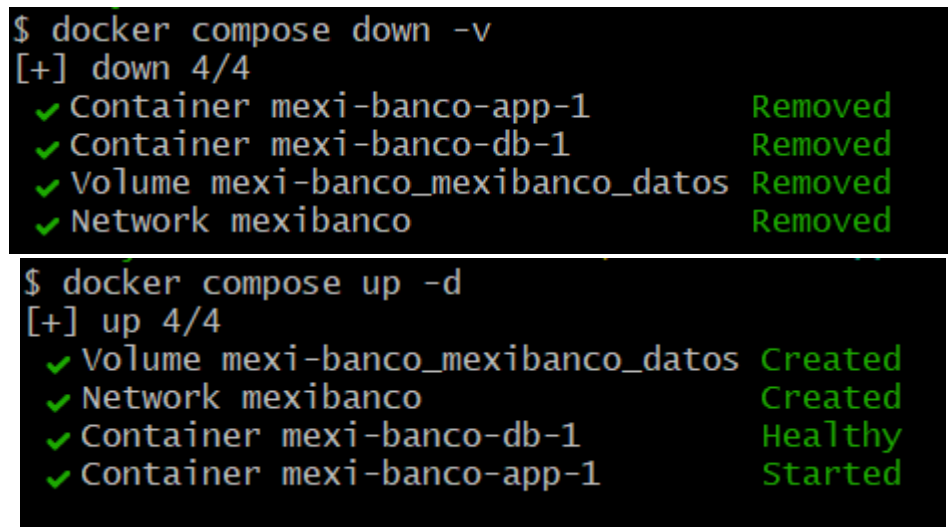
$ docker compose up -d
[+] up 3/3
✓ Network mexibanco Created
✓ Container mexi-banco-db-1 Healthy
✓ Container mexi-banco-app-1 Started
```



```
Body Cookies Headers 5 Test Results 2/2 200 OK
[{} JSON] Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180791160757715",
4   "titular": "Ana",
5   "saldo": 800.00
```

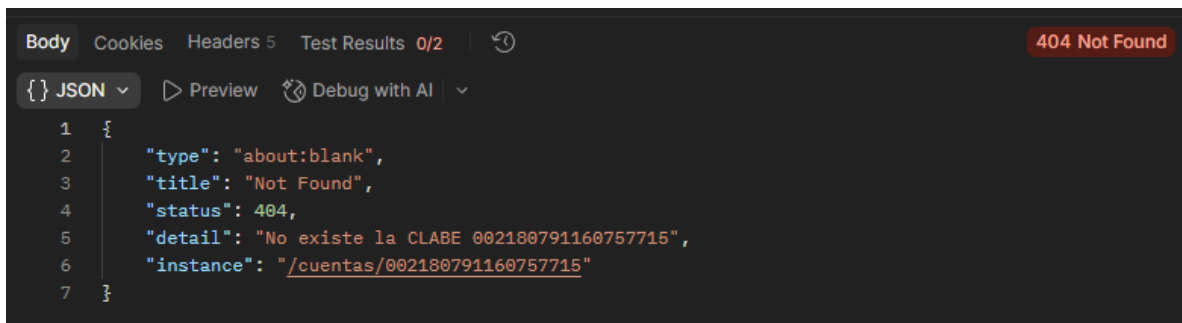
[Consultar origen](#)

Al ejecutar `docker compose down -v`, la cuenta que creé desapareció. La bandera `-v` le indicó a Docker que también debía destruir los volúmenes asociados, lo que borró permanentemente el estado y la información de la base de datos.



```
$ docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1      Removed
✓ Container mexi-banco-db-1      Removed
✓ Volume mexi-banco_mexibanco_datos Removed
✓ Network mexibanco              Removed

$ docker compose up -d
[+] up 4/4
✓ Volume mexi-banco_mexibanco_datos Created
✓ Network mexibanco              Created
✓ Container mexi-banco-db-1      Healthy
✓ Container mexi-banco-app-1     Started
```



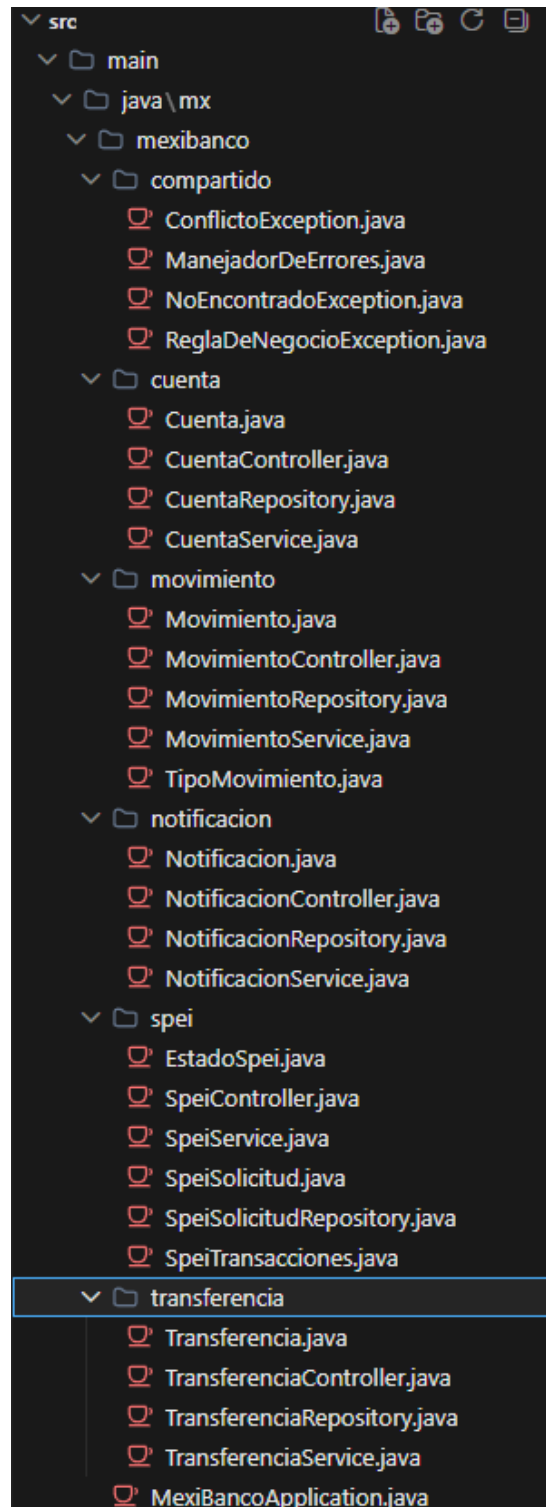
```
Body Cookies Headers 5 Test Results 0/2 404 Not Found
[{} JSON] Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 002180791160757715",
6   "instance": "/cuentas/002180791160757715"
7 }
```

[Consultar origen](#)

Paso 5: Revisión del Código

Al explorar el código fuente del proyecto, pude comprobar que se trata de una aplicación monolítica escrita en Java. El sistema está organizado en cinco módulos principales: cuenta, movimiento, notificación, spei y transferencia.

Revisando el interior de cada uno (por ejemplo, el módulo cuenta o transferencia), identifiqué claramente la arquitectura de cuatro capas, ya que cada módulo cuenta con sus respectivos archivos de entidad (Cuenta.java), repositorio (CuentaRepository.java), servicio (CuentaService.java) y controlador (CuentaController.java). Además, noté que existe un módulo compartido encargado de manejar las excepciones (como ConflictoException o NoEncontradoException) y los errores globales.



Paso 6: Estudio del compose y el Dockerfile

Analicé el archivo y observé que utiliza un patrón de construcción en dos etapas para optimizar el tamaño final de la imagen. Utiliza la imagen `eclipse-temurin:21-jdk-alpine` (que incluye el JDK completo) para descargar las dependencias de Maven y compilar el código fuente en un archivo `.jar`. Para la ejecución utiliza una imagen mucho más ligera, `eclipse-temurin:21-jre-alpine` (solo con el JRE). En esta etapa simplemente se copia el `.jar` compilado de la etapa anterior, se expone el puerto 8080 y se define el comando de arranque de la aplicación.

```
# ---- ETAPA 1: CONSTRUCCION ----  
FROM eclipse-temurin:21-jdk-alpine AS builder  
WORKDIR /app  
COPY pom.xml .  
COPY mvnw .  
COPY .mvn .mvn  
RUN ./mvnw dependency:go-offline -B  
COPY src ./src  
RUN ./mvnw clean package -DskipTests  
  
# ---- ETAPA 2: EJECUCION ----  
FROM eclipse-temurin:21-jre-alpine  
WORKDIR /app  
COPY --from=builder /app/target/*.jar app.jar  
EXPOSE 8080  
  
ENTRYPOINT ["java", "-jar", "app.jar"]
```